

Implementasi Algoritma Merge Sort Berbasis Divide and Conquer untuk Pengurutan Data Nilai Akademik Mahasiswa pada Sistem Informasi Akademik Universitas

Sahara Lani Lestari^a, Frengki Alfredo Matondang^b, Dinda Syafitri^c, Kayla Amelia Putri^d, Adidtya Perdana^e

^aIlmu Komputer, Fakultas Matematika dan Ilmu Pengetahuan Alam, Universitas Negeri Medan, Lestarisahara28@gmail.com

^bIlmu Komputer, Fakultas Matematika dan Ilmu Pengetahuan Alam, Universitas Negeri Medan, fr4nkelblue.689@gmail.com

^cIlmu Komputer, Fakultas Matematika dan Ilmu Pengetahuan Alam, Universitas Negeri Medan, Dindasyafitri06@gmail.com

^dIlmu Komputer, Fakultas Matematika dan Ilmu Pengetahuan Alam, Universitas Negeri Medan, kaylaamelia499@gmail.com

^eIlmu Komputer, Fakultas Matematika dan Ilmu Pengetahuan Alam, Universitas Negeri Medan, adidtya@unimed.ac.id

Abstrak

Penelitian ini mengkaji penerapan algoritma Merge Sort dalam proses pengurutan data nilai akademik mahasiswa pada sistem informasi akademik (SIKAD). Permasalahan yang diangkat adalah belum optimalnya pengolahan data akademik yang berpotensi menimbulkan keterlambatan serta kesalahan dalam penyajian informasi. Tujuan penelitian ini adalah untuk mengimplementasikan serta mengevaluasi kinerja algoritma Merge Sort dengan membandingkannya terhadap Bubble Sort dan Insertion Sort. Metode yang digunakan adalah pendekatan eksperimental melalui pengujian pada berbagai ukuran dataset, mulai dari skala kecil hingga besar, serta pada kondisi data yang berbeda, yaitu acak, terurut, dan terbalik. Implementasi dilakukan melalui aplikasi berbasis Command Line Interface (CLI) dan antarmuka web sebagai simulasi penggunaan nyata. Hasil penelitian menunjukkan bahwa Merge Sort memiliki performa yang lebih efisien dan konsisten dibandingkan algoritma lainnya, khususnya pada dataset berukuran besar. Selain itu, algoritma ini memiliki sifat *stable sort* yang mampu mempertahankan urutan relatif data dengan nilai yang sama, sehingga lebih andal dalam pengolahan data akademik.

Kata Kunci: Merge Sort, Divide and Conquer, Algoritma Pengurutan, Sistem Informasi Akademik (SIKAD), Analisis Performa

Abstract

This study examines the application of the Merge Sort algorithm in the process of sorting student academic data within an academic information system (SIKAD). The issue addressed is the suboptimal processing of academic data, which has the potential to cause delays and errors in the presentation of information. The objective of this study is to implement and evaluate the performance of the Merge Sort algorithm by comparing it with Bubble Sort and Insertion Sort. The method used is an experimental approach through testing on various dataset sizes, ranging from small to large scales, as well as under different data conditions, namely random, sorted, and reversed. Implementation was carried out using a Command Line Interface (CLI)-based application and a web interface to simulate real-world usage. The results of the study indicate that Merge Sort performs more efficiently and consistently than other algorithms, particularly on large datasets. Additionally, this algorithm possesses stable sort properties that maintain the relative order of data with the same values, making it more reliable for academic data processing.

Keywords: Merge Sort, Divide and Conquer, Sorting Algorithms, Academic Information System (SIKAD), Performance Analysis

This work is licensed under Creative Commons Attribution License 4.0 CC-BY International license



PENDAHULUAN

Perkembangan pesat di bidang Teknologi Informasi (TI) telah mendorong kebutuhan mendesak akan sistem manajemen data yang cepat, akurat, dan terintegrasi di berbagai sektor [1]. Hal ini memiliki dampak yang tidak dapat diabaikan pada dunia Pendidikan [2]. Perguruan tinggi mulai menggunakan teknologi informasi untuk mendukung kegiatan akademik dan administrasi karena kebutuhan akan layanan yang cepat, tepat, dan transparan. Sistem Informasi Akademik (SIKAD) adalah salah satu contoh nyata dari pemanfaatan teknologi tersebut [3]. Sistem yang menghasilkan layanan data dan informasi akademik disebut Sistem Informasi Akademik (SIKAD). Sistem ini dirancang untuk menyesuaikan proses bisnis berkelanjutan dengan tujuan untuk mempermudah dan meningkatkan kualitas layanan akademik. Tujuan SIKAD adalah untuk menjadikan pengelolaan data akademik menjadi efektif dan efisien. Sistem ini juga mendukung analisis data untuk pengambilan keputusan kampus [4].

Namun, Pengolahan data nilai yang masih dilakukan secara konvensional telah menunjukkan bahwa tidak hanya memakan waktu yang lama, tetapi juga rentan terhadap kesalahan pencatatan dan kehilangan data, serta kesulitan mencari dan menggabungkan data nilai siswa. Akibatnya, proses pelaporan dan pencarian data menjadi lebih sulit [5]. Oleh karena itu, Teknologi Informasi (TI) memegang peranan penting dalam proses pengolahan data dan pengelolaan sistem informasi. Pemanfaatan TI telah memberikan dampak besar terhadap pengelolaan data mahasiswa, proses pencatatan, serta pengolahan informasi menjadi lebih efektif dan terstruktur [6].

Salah satu komponen penting dari sistem informasi akademik program studi adalah pengelolaan data siswa. Tempat data dapat ditemukan dengan cepat, tepat, dan akurat. Nama, NIM, semester, status keaktifan, dan informasi lainnya diperlukan untuk mahasiswa [7]. Dalam SIKAD, berbagai data penting tersebut disimpan sehingga

memerlukan pengelolaan yang sistematis [8]. Karena data yang disusun dengan baik akan lebih mudah diakses, dianalisis, dan dipahami sehingga membantu pengambilan keputusan yang lebih baik, proses pengurutan data menjadi bagian yang tidak dapat diabaikan seiring dengan meningkatnya volume data. Proses pengurutan data sangat penting untuk berbagai kegiatan akademik, seperti menyusun peringkat siswa, melakukan analisis prestasi siswa, dan menyajikan laporan siswa secara sistematis. Oleh karena itu, algoritma pengurutan (sorting) juga dianggap sebagai solusi yang tepat untuk menangani masalah pengelolaan data siswa secara optimal dan efisien [9]. Pengurutan atau “Sorting” adalah suatu proses menyusun kembali data yang sebelumnya telah tersusun secara acak hingga tersusun secara teratur menurut aturan tertentu [10].

Algoritma pengurutan (sorting) juga merupakan salah satu konsep penting dalam ilmu yang banyak digunakan dalam berbagai bidang, seperti pengolahan data, pencarian informasi, basis data, dan pengembangan perangkat lunak. Mengurutkan data memungkinkan proses pencarian dan pengolahan informasi menjadi lebih cepat dan efisien [11]. Seperti penelitian yang dilakukan oleh Pratama et al., (2024) telah menunjukkan bahwa algoritma pengurutan dapat diterapkan pada data akademik. Salah satu contohnya adalah penggunaan Sort Selection untuk mengurutkan data siswa berdasarkan nama, NIM, kelas, dan semester, yang menghasilkan hasil pengaturan yang benar dan konsisten baik secara naik maupun turun [12] dan penelitian yang dilakukan oleh Wijaya et al., (2025) menunjukkan bahwa menggunakan algoritma pengurutan pada data nilai siswa membuat peringkat dan penyampaian statistik akademik lebih mudah yang sebelumnya dilakukan secara manual dan berpotensi menyebabkan kesalahan [13].

Menurut Tumanggor et al., (2022) Salah satu algoritma yang dapat digunakan untuk melakukan proses pengurutan data Adalah algoritma Merge Sort. Merge Sort merupakan salah satu algoritma pengurutan yang paling banyak digunakan, yang bekerja berdasarkan prinsip Divide and Conquer. Metode divide and conquer membagi suatu masalah besar menjadi beberapa masalah yang lebih kecil dan lebih mudah diselesaikan. Kemudian, solusi dari masing-masing submasalah digabungkan kembali untuk menghasilkan solusi terakhir untuk masalah utama [14]. Memiliki kompleksitas waktu $O(n \log n)$, algoritma Merge Sort lebih efisien dalam mengolah dataset yang lebih besar dan kompleks daripada algoritma pengurutan yang lebih sederhana [15].

Berbagai penelitian yang telah dilakukan seperti penelitian oleh Syahputra (2022) algoritma Selection Sort digunakan untuk mengurutkan data IPK siswa secara terstruktur dari nilai tertinggi hingga nilai terendah. Hasil penelitian menunjukkan bahwa algoritma ini memiliki kemampuan mengurutkan data dengan baik, tetapi memiliki kompleksitas waktu $O(n^2)$, sehingga tidak efektif untuk dataset yang sangat besar [16]. Serta penelitian oleh Abdullah et al., (2023) yang menggunakan algoritma Sort Bubble untuk mengurutkan data Nomor Induk Mahasiswa (NIM). Hasil penelitian menunjukkan bahwa algoritma ini efektif untuk pengurutan, tetapi memiliki kompleksitas waktu $O(n^2)$. Akibatnya, algoritma ini juga tidak efisien untuk mengurutkan sejumlah besar data [17].

Berdasarkan permasalahan tersebut, diperlukan penerapan algoritma pengurutan yang lebih efisien dalam pengelolaan data akademik. Oleh karena itu Penelitian ini mengurutkan data nilai akademik siswa dalam sistem informasi akademik dengan algoritma Merge Sort dan pendekatan Divide and Conquer. Ini menunjukkan bahwa algoritma pengurutan yang lebih efisien diperlukan untuk mengelola data akademik. Pendekatan ini diharapkan menjadikan proses pengolahan data lebih cepat dan efisien, dan mampu meningkatkan kualitas penyajian data akademik.

METODE PENELITIAN

Jenis dan Pendekatan Penelitian

Penelitian ini menggunakan pendekatan eksperimental berbasis rekayasa perangkat lunak (*software engineering*). Jenis penelitian yang diterapkan adalah penelitian terapan (*applied research*) yang berfokus pada implementasi dan evaluasi algoritma Merge Sort berbasis Divide and Conquer untuk menyelesaikan permasalahan pengurutan data nilai akademik mahasiswa pada Sistem Informasi Akademik (SIA) Universitas. Penelitian ini bersifat kuantitatif karena mengukur dan membandingkan performa algoritma berdasarkan parameter yang dapat dikuantifikasi, yaitu waktu eksekusi (*running time*) dalam satuan milidetik pada berbagai ukuran dan kondisi dataset.

Alat dan Bahan Penelitian

Perangkat keras yang digunakan dalam penelitian ini adalah laptop atau PC dengan spesifikasi minimum prosesor Intel Core i3 atau setara, RAM 4 GB, dan penyimpanan 256 GB, dilengkapi dengan koneksi internet untuk keperluan akses referensi dan dokumentasi. Perangkat lunak yang digunakan meliputi bahasa pemrograman Python 3.x sebagai bahasa utama implementasi algoritma, *Integrated Development Environment* (IDE) Visual Studio Code untuk pengembangan dan pengujian kode, serta beberapa library Python yaitu *random* untuk pembangkitan dataset, *time* untuk pengukuran waktu eksekusi, dan *matplotlib* untuk visualisasi grafik perbandingan kinerja algoritma.

Populasi dan Sampel Data

Data yang digunakan dalam penelitian ini adalah data sintesis (*synthetic data*) yang dibangkitkan secara acak menggunakan program komputer, merepresentasikan data nilai akademik mahasiswa. Setiap record data mencakup atribut NIM, Nama Mahasiswa, Program Studi, Mata Kuliah, Nilai (skala 0–100), Grade, dan IPK (skala 0,00–4,00).

Penggunaan data sintetis dipilih untuk menjaga privasi data mahasiswa sekaligus memungkinkan pengujian pada berbagai skala data yang dikehendaki.

Ukuran sampel yang digunakan dalam pengujian performa bervariasi, yaitu 100, 500, 1.000, 5.000, 10.000, 50.000, dan 100.000 record data. Variasi ukuran sampel ini bertujuan untuk menguji performa dan skalabilitas masing-masing algoritma pada berbagai kondisi beban data yang merepresentasikan kondisi nyata sistem informasi akademik perguruan tinggi dari skala kecil hingga besar.

Metode Pengembangan Sistem

Pengembangan prototipe sistem dalam penelitian ini menggunakan metode *Waterfall* yang dimodifikasi dengan empat tahapan utama. Pertama, tahap Analisis Kebutuhan, yaitu identifikasi kebutuhan fungsional dan non-fungsional sistem yang meliputi kemampuan pengurutan data, pengukuran waktu eksekusi, dan penyajian hasil perbandingan. Kedua, tahap Perancangan, yaitu pembuatan pseudocode, flowchart, dan desain struktur data algoritma Merge Sort, Bubble Sort, dan Insertion Sort. Ketiga, tahap Implementasi, yaitu penerjemahan rancangan ke dalam kode program Python. Keempat, tahap Pengujian, yaitu uji kebenaran (*correctness testing*) dan uji performa (*performance testing*) pada seluruh skenario yang telah ditentukan.

Prosedur Penelitian

Prosedur penelitian dilaksanakan secara sistematis melalui delapan langkah. Langkah pertama adalah studi literatur mengenai algoritma Merge Sort, paradigma Divide and Conquer, dan kompleksitas algoritma pengurutan. Langkah kedua adalah perancangan pseudocode dan flowchart algoritma Merge Sort. Langkah ketiga adalah implementasi algoritma Merge Sort, Bubble Sort, dan Insertion Sort menggunakan bahasa pemrograman Python. Langkah keempat adalah pembangkitan dataset sintetis berupa data nilai akademik mahasiswa dalam berbagai ukuran. Langkah kelima adalah pengujian kebenaran untuk memastikan output pengurutan sudah tepat dan valid. Langkah keenam adalah pengujian performa dengan mengukur waktu eksekusi pada setiap skenario, yaitu data acak (*random*), data terurut (*sorted*), dan data terurut terbalik (*reverse sorted*). Langkah ketujuh adalah pencatatan dan tabulasi data hasil pengujian. Langkah kedelapan adalah analisis dan interpretasi data hasil pengujian untuk menarik kesimpulan.

Teknik Analisis Data

Analisis data dalam penelitian ini dilakukan melalui dua pendekatan yang saling melengkapi. Pendekatan pertama adalah analisis kompleksitas teoritis menggunakan notasi Big-O untuk menghitung kompleksitas waktu dan ruang secara matematis pada kondisi terbaik (*best case*), rata-rata (*average case*), dan terburuk (*worst case*). Pendekatan kedua adalah analisis eksperimental dengan menjalankan setiap skenario pengujian sebanyak lima kali iterasi, kemudian merata-ratakan hasilnya untuk mendapatkan nilai yang representatif dan mengurangi pengaruh fluktuasi performa sistem. Hasil pengukuran disajikan dalam bentuk tabel perbandingan waktu eksekusi dan grafik visualisasi untuk memudahkan interpretasi dan penarikan kesimpulan. Perbandingan kompleksitas teoritis dari ketiga algoritma yang diuji disajikan pada Tabel 1 berikut.

Algoritma	Best Case	Average Case	Worst Case	Space Complexity
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$

Tabel 1. Perbandingan Kompleksitas Teoritis Algoritma Sorting

Variabel Penelitian

Variabel bebas dalam penelitian ini meliputi jenis algoritma pengurutan (Merge Sort, Bubble Sort, dan Insertion Sort), ukuran dataset (n), serta kondisi awal data (acak, terurut, dan terurut terbalik). Variabel terikat adalah waktu eksekusi algoritma dalam satuan milidetik dan kebenaran hasil pengurutan. Variabel kontrol meliputi spesifikasi perangkat keras yang digunakan selama pengujian serta versi Python yang digunakan, yaitu Python 3.x, guna memastikan konsistensi dan validitas hasil perbandingan antar algoritma.

HASIL DAN PEMBAHASAN

Implementasi Sistem

Aplikasi berbasis *Command Line Interface* (CLI) yang dikembangkan dalam penelitian ini memiliki tiga fitur utama yang mendukung pengolahan serta analisis data. Fitur pertama adalah pengelolaan data, yang berfungsi menampilkan 28 data mahasiswa dalam bentuk tabel terstruktur serta menyediakan fasilitas pengurutan berdasarkan NIM dan nama secara *ascending*, serta IPK secara *descending* menggunakan algoritma Merge Sort. Sistem juga memungkinkan penambahan data sintetis untuk memperluas kebutuhan pengujian.

Fitur kedua adalah benchmark dan analisis performa, yang digunakan untuk menguji kinerja tiga algoritma pengurutan pada berbagai ukuran dataset (100 hingga 20.000 data) dan tiga kondisi awal, yaitu acak, terurut, dan terbalik. Hasil pengujian ditampilkan dalam bentuk tabel serta divisualisasikan melalui grafik *matplotlib* secara *real-time*. Fitur ketiga adalah visualisasi animasi algoritma, yang menampilkan proses kerja Merge Sort dalam bentuk grafik batang dinamis, sehingga mempermudah pemahaman tahapan *divide*, *conquer*, dan *merge*.

Antarmuka web SIAKAD-M dikembangkan sebagai prototipe sistem informasi akademik berbasis HTML5, Tailwind CSS, JavaScript, dan Chart.js yang bersifat interaktif. Pada tab Dashboard & Data, sistem menyajikan statistik mahasiswa serta tabel interaktif yang mendukung pengurutan data secara langsung menggunakan Merge Sort dengan waktu eksekusi yang ditampilkan secara *real-time*. Fitur tambahan seperti pencarian, filter, dan ekspor data juga disediakan. Pada tab Analisis Performa, sistem menampilkan perbandingan kinerja algoritma dalam bentuk grafik interaktif. Sementara itu, tab Animasi Algoritma menyajikan visualisasi langkah demi langkah proses Merge Sort menggunakan grafik batang berwarna untuk memperjelas alur pengurutan.

Algoritma Merge Sort yang digunakan menerapkan pendekatan *divide and conquer*, yaitu membagi data menjadi bagian yang lebih kecil, menyelesaikannya secara rekursif, kemudian menggabungkannya kembali hingga menghasilkan data yang terurut. Pseudocode algoritma Merge Sort yang diimplementasikan dalam penelitian ini adalah sebagai berikut

```
FUNGSI merge_sort(arr, key, direction):
  JIKA len(arr) <= 1 MAKA
    KEMBALIKAN arr // Base case: array 1 elemen sudah terurut
  mid ← len(arr) // 2 // Titik tengah pembagian
  left ← merge_sort(arr[0..mid], key, direction) // Rekursif kiri
  right ← merge_sort(arr[mid..n], key, direction) // Rekursif kanan
  KEMBALIKAN merge(left, right, key, direction)

FUNGSI merge(left, right, key, direction):
  result ← [], i ← 0, j ← 0
  SELAMA i < len(left) DAN j < len(right):
    JIKA direction='asc' DAN left[i][key] <= right[j][key]:
      Tambah left[i] ke result; i ← i + 1
    SEBALIKNYA:
      Tambah right[j] ke result; j ← j + 1
  Tambah sisa elemen left[i:] dan right[j:] ke result
  KEMBALIKAN result
```

Gambar 1. Pseudocode Algoritma Merge Sort

Hasil Pengujian Kebenaran (Correctness Testing)

Skenario Uji	Input Data	Kondisi Awal	Output yang Diharapkan	Hasil Pengujian
TC-01	28 record mahasiswa	Data acak (random)	Terurut ascending berdasarkan NIM	LULUS — Output terurut benar
TC-02	28 record mahasiswa	Data acak (random)	Terurut ascending berdasarkan Nama	LULUS — Output terurut benar
TC-03	28 record mahasiswa	Data acak (random)	Terurut descending berdasarkan IPK	LULUS — Ranking beasiswa akurat
TC-04	28 record mahasiswa	Data sudah terurut	Output identik (stabilitas)	LULUS — Urutan relatif dipertahankan
TC-05	28 record mahasiswa	Data terurut terbalik	Terurut penuh ascending	LULUS — Hasil benar dalam semua kondisi
TC-06	1 record saja	Elemen tunggal	Output sama dengan input	LULUS — Base case ditangani dengan benar
TC-07	Nilai IPK sama (3.80)	Data acak	Urutan NIM dipertahankan (stabil)	LULUS — Sifat stable sort terbukti

Tabel 1. Hasil Pengujian Kebenaran (Correctness Testing)

Untuk memastikan bahwa algoritma Merge Sort mampu menghasilkan output yang sesuai dengan spesifikasi pada berbagai kondisi data, pengujian kebenaran dilakukan. Hasilnya menunjukkan bahwa seluruh skenario pengujian menunjukkan status LULUS, yang menunjukkan bahwa algoritma ini sangat akurat dan mampu menangani banyak variasi input secara konsisten. Selain itu, hasil pengujian pada skenario TC-07 menunjukkan bahwa algoritma ini memiliki karakteristik sort stabil, yang berarti bahwa itu dapat mempertahankan urutan relatif antar data dengan nilai kunci yang sama. Ini adalah keunggulan yang signifikan, terutama dalam pengolahan data akademik yang sering memiliki nilai yang sama.

Hasil Pengujian Performa

Pengujian performa dilakukan dengan menghitung waktu eksekusi ketiga algoritma pada berbagai ukuran dataset dan kondisi awal data. Setiap skenario pengujian diulang 17 kali sebanyak 3 iterasi, dan kemudian hasilnya dirata-ratakan untuk mendapatkan nilai yang lebih representatif. Hasilnya adalah sebagai berikut:

- Kondisi Data Acak (Average Case / Random)**

Kondisi data acak adalah situasi yang paling sering terjadi dalam pengelolaan data akademik, di mana data siswa tidak diatur dalam urutan tertentu.

n (Ukuran Data)	Merge Sort (ms)	Bubble Sort (ms)	Insertion Sort (ms)	Rasio Keunggulan
100	0.179	0.873	0.344	Bubble 4.9x, Insertion 1.9x lebih lambat
500	10.572	15.771	7.431	Merge konsisten, Bubble 1.5x lebih lambat
1.000	2.671	86.865	36.721	Bubble 32.5x, Insertion 13.7x lebih lambat
5.000	19.004	2.449.517	1.033.286	Merge unggul signifikan (130x vs Bubble)
10.000	43.324	10.704.277	4.878.885	Merge 247x lebih cepat dari Bubble

Tabel 2 Perbandingan Waktu Eksekusi — Kondisi Data Acak (ms)

Algoritma Merge Sort menunjukkan kinerja yang lebih baik daripada Bubble Sort dan Insertion Sort dalam kondisi data acak, seperti yang ditunjukkan dalam Tabel 2. Dengan bertambahnya ukuran dataset, perbedaan waktu eksekusi semakin terlihat. Namun, Merge Sort masih dapat menyelesaikan proses dengan waktu yang relatif singkat. Ini menunjukkan bahwa Merge Sort memiliki skalabilitas yang baik ketika menangani data yang sangat besar. Keunggulan ini sejalan dengan kompleksitas waktunya, yaitu $O(n \log n)$, yang membuatnya lebih efisien daripada algoritma lain dengan kompleksitas $O(n^2)$.

- Kondisi Data Terurut (Best Case / Sorted)**

Kondisi data terurut menggambarkan situasi di mana data mahasiswa telah berada dalam keadaan tersusun sebelumnya, misalnya ketika sistem melakukan proses pengurutan kembali terhadap data yang sudah terurut dari penggunaan sebelumnya.

n (Ukuran Data)	Merge Sort (ms)	Bubble Sort (ms)	Insertion Sort (ms)	Rasio Keunggulan
100	0.130	0.656	0.437	Merge stabil, Bubble 5.0x lebih lambat
500	0.743	18.011	14.081	Merge unggul 24x dari Bubble
1.000	1.588	91.309	62.417	Merge unggul 57x dari Bubble
5.000	9.868	3.075.225	2.312.121	Merge unggul 312x dari Bubble
10.000	22.770	13.904.501	10.389.488	Merge unggul 611x dari Bubble

Tabel 3. Perbandingan Waktu Eksekusi — Kondisi Data Terurut (ms)

Pada Tabel 3, Merge Sort tetap menunjukkan kinerja yang stabil meskipun data sudah dalam kondisi terurut, meskipun waktu eksekusinya meningkat secara proporsional, tetapi masih lebih lambat daripada Bubble Sort dan Insertion Sort, terutama untuk data berukuran besar. Hasil pengujian menunjukkan bahwa Merge Sort masih belum mampu menyaingi algoritma lain dalam hal kecepatan data terurut, menunjukkan bahwa Merge Sort tidak bergantung pada kondisi awal data dan lebih konsisten dalam berbagai kondisi.

- Kondisi Data Terbalik (Worst Case / Reversed)**

Kondisi data terurut terbalik menunjukkan kondisi terburuk ketika data berada dalam urutan yang berlawanan dari yang diinginkan. Ini terjadi dalam kasus di mana data terurut naik tetapi dibutuhkan dalam urutan turun.

n (Ukuran Data)	Merge Sort (ms)	Bubble Sort (ms)	Insertion Sort (ms)	Rasio Keunggulan
100	0.132	0.010	0.012	Bubble & Insertion unggul (data sudah hampir terurut)

500	0.678	0.053	0.070	Keunggulan Bubble pada reversed kecil
1.000	1.477	0.105	0.148	Bubble sangat cepat pada data reversed kecil
5.000	9.156	0.810	1.469	Merge mulai setara pada n besar
10.000	25.017	2.135	2.384	Bubble cepat karena early termination

Tabel 4. Perbandingan Waktu Eksekusi — Kondisi Data Terurut Terbalik (ms)

Tabel 4 menunjukkan bahwa Bubble Sort dan Insertion Sort bekerja lebih cepat pada ukuran data yang lebih kecil dalam kondisi terbalik. Mekanisme optimasi seperti early termination, yang memungkinkan proses berhenti lebih awal, merupakan hasilnya. Namun, keunggulan tersebut tidak berlaku untuk ukuran data yang lebih besar, di mana Merge Sort menunjukkan kinerja yang lebih buruk lagi. Selain itu, kondisi data terbalik jarang terjadi dalam praktik nyata, sehingga keunggulan tersebut tidak terlalu penting untuk diterapkan dalam sistem akademik.

Catatan: Karena implementasinya menggunakan flag swapped untuk early termination, Bubble Sort sangat cepat dalam kondisi data terbalik. Ketika data sudah dalam keadaan terbalik sempurna dan diurutkan ke arah yang sama dengan kondisi terbalik, Bubble Sort dapat menyelesaikan pengurutan dengan sedikit iterasi. Keunggulan ini, bagaimanapun, tidak dapat diandalkan karena bergantung pada kondisi data acak dunia nyata.

• Grafik Perbandingan Kinerja

Berdasarkan data pengujian, ringkasan perbandingan kinerja ketiga algoritma disajikan dalam tabel rekapitulasi berikut:

Kondisi Data	Merge Sort	Bubble Sort	Insertion Sort
Random (Average)	43.324 ms	10.704.277 ms	4.878.885 ms
Sorted (Best)	22.770 ms	13.904.501 ms	10.389.488 ms
Reversed (Worst)	25.017 ms	2.135 ms	2.384 ms

Tabel 5. Rekapitulasi Kinerja Algoritma pada $n=10.000$

Merge Sort secara konsisten memberikan performa terbaik pada kondisi data urut dan acak, seperti yang ditunjukkan dalam Tabel 5. Ini menunjukkan bahwa algoritma ini lebih efisien dan dapat diandalkan untuk pengolahan data akademik yang biasanya dinamis. Namun, Bubble Sort dan Insertion Sort menunjukkan kinerja yang kurang efisien, terutama untuk dataset berukuran besar. Hasil ini menunjukkan bahwa Merge Sort adalah pilihan algoritma yang paling tepat karena kinerjanya stabil dan tidak bergantung pada kondisi awal data.

Hasil pengujian menunjukkan bahwa algoritma Sort Merge memiliki keunggulan yang signifikan dalam hal konsistensi dan efisiensi kinerja dibandingkan dengan Sort Bubble dan Sort Insertion, terutama dalam hal dataset yang sangat besar. Merge Sort mampu menyelesaikan proses pengurutan dengan waktu yang jauh lebih cepat dalam kondisi data acak, yang merupakan situasi paling mungkin dalam pengelolaan data akademik. Perbedaan kinerja ini semakin terlihat seiring bertambahnya jumlah data; algoritma dengan kompleksitas $O(n^2)$ mengalami peningkatan yang signifikan dalam waktu eksekusi, sedangkan Merge Sort dengan kompleksitas $O(n \log n)$ tetap memiliki kinerja yang stabil dan skalabel. Merge Sort tidak bergantung pada karakteristik awal data, tetap konsisten bahkan dalam kondisi data yang telah terur

Bubble Sort dapat bekerja lebih cepat dalam beberapa situasi tertentu, seperti data terurut terbalik (reversed). Namun, keunggulan ini bersifat situasional dan tidak mencerminkan kondisi data dalam implementasi nyata, yang biasanya acak atau semi-terurut. Akibatnya, itu tidak memiliki dampak yang signifikan pada sistem akademik. Selain keunggulan dalam hal performa, Merge Sort juga memiliki keunggulan dalam hal kestabilan (stable sort), yaitu kemampuan untuk mempertahankan urutan data yang sebanding dengan nilai kunci yang sama. Ini sangat penting untuk sistem informasi akademik, terutama untuk proses pemeringkatan siswa, karena mampu menjaga hasil pengurutan konsisten saat nilai atau IPK tetap sama.

Selain itu, implementasi algoritma Merge Sort pada sistem SIAKAD-M menunjukkan hasil yang efektif, karena memungkinkan proses pengurutan data siswa dengan sangat cepat pada skala kecil. Sebagai prototipe sistem informasi akademik yang tidak hanya efisien tetapi juga informatif dan mudah digunakan, fungsi sistem semakin diperkuat dengan dukungan fitur interaktif seperti pencarian, filter, ekspor data, dan visualisasi algoritma. Secara keseluruhan, temuan penelitian ini menunjukkan bahwa algoritma Merge Sort berbasis divide and conquer adalah solusi yang tepat dan dapat diandalkan untuk pengurutan data akademik, terutama ketika dihadapkan pada volume data yang signifikan dan kebutuhan akan hasil yang konsisten dan akurat.

SIMPULAN

Berdasarkan hasil penelitian, penerapan algoritma Merge Sort dengan pendekatan Divide and Conquer dalam proses pengurutan data nilai akademik pada sistem informasi akademik (SIKAD) terbukti mampu meningkatkan efisiensi serta keandalan dalam pengolahan data. Hasil pengujian pada berbagai ukuran dan kondisi data (acak, terurut, dan terbalik) menunjukkan bahwa Merge Sort memiliki kinerja yang stabil dan konsisten dengan kompleksitas waktu $O(n \log n)$. Jika dibandingkan dengan Bubble Sort dan Insertion Sort yang memiliki kompleksitas $O(n^2)$, Merge Sort lebih unggul terutama dalam menangani dataset berukuran besar karena waktu prosesnya jauh lebih cepat dan tidak meningkat secara signifikan.

Selain itu, kelebihan lain dari Merge Sort adalah sifatnya sebagai stable sort, yang mampu menjaga urutan relatif data dengan nilai yang sama, sehingga sangat sesuai digunakan dalam kebutuhan akademik seperti proses pemeringkatan mahasiswa. Implementasi algoritma ini melalui aplikasi berbasis CLI dan web juga menunjukkan bahwa Merge Sort dapat diterapkan secara optimal dalam sistem nyata, dengan dukungan fitur analisis dan visualisasi yang mempermudah pengguna. Oleh karena itu, Merge Sort dapat dijadikan sebagai solusi yang efektif, akurat, dan dapat diandalkan dalam pengolahan serta pengurutan data akademik pada berbagai skala.

DAFTAR PUSTAKA

- [1] M. A. Putri, N. Ardiningtyas, A. Firzatullah, R. Basatha, and N. D. Septiyanti, "Rancang Bangun Sistem Manajemen Aset Sekolah ' S-Inventory ' Berbasis Desktop," *J. Ilmu Komput. Dan Inform.*, vol. 2, no. 2, pp. 43–51, 2025.
- [2] U. H. Salsabila and N. Agustian, "PERAN TEKNOLOGI PENDIDIKAN DALAM PEMBELAJARAN Unik," *Islam. J. Keislam. dan Ilmu Pendidik.*, vol. 3, no. 1, pp. 123–133, 2021.
- [3] F. Rahman, P. B. Arti, H. Yeni, U. M. Mamuju, U. M. Mamuju, and U. M. Mamuju, "Pengaruh penerapan sistem informasi akademik terhadap kualitas layanan administrasi kampus," *JIMAKUN J. Ilmu Manaj. dan Akunt. Nusan.*, vol. 02, no. 1, pp. 48–53, 2026.
- [4] S. N. Oktaviana, V. Apriliani, and W. N. Novita, "Implementasi Sistem Informasi Akademik Dalam Meningkatkan Mutu Pelayanan Kampus," *J. Soshum Insentif*, vol. 7, no. 1, pp. 53–63, 2024.
- [5] N. Nafisah, M. H. Ujianti, and Riyanto, "SISTEM INFORMASI AKADEMIK PENGOLAHAN DATA NILAI SISWA BERBASIS WEBSITE PADA SD NEGERI MANGUNSAREN 02," *JATI (Jurnal Mhs. Tek. Inform.*, vol. 9, no. 4, pp. 6648–6654, 2025.
- [6] N. Noviyana and M. I. P. Nasution, "Penerapan Teknologi Informasi Untuk Efektivitas dan Efisiensi Pengolahan Data Mahasiswa," *J. Ilm. Res. Dev. Student*, vol. 2, no. 1, pp. 152–160, 2024.
- [7] F. Pratama, D. Juniardi, M. F. Arif, and Suharsono, "Implementasi algoritma selection sort untuk pengurutan data mahasiswa di program studi teknik informatika politeknik negeri pontianak," *Pros. Semin. Nas. Sains dan Teknol. Seri 02 Fak. Sains dan Teknol. Univ. Terbuka*, vol. 1, no. 2, pp. 282–292, 2024.
- [8] A. Ghofur, J. Nazila, N. Holidiyah, and S. Kholifah, "Penerapan Algoritma Insertion Sort Pada Aplikasi Pengolahan Data Mahasiswa Menggunakan Java Di Fakultas Sains Dan Teknologi Prodi Teknologi Informasi," *Eastasouth J. Posit. Community Serv.*, vol. 4, no. 01, pp. 12–19, 2025, doi: 10.58812/ejpcs.v4i01.
- [9] M. Aditya, H. Zenitha, S. Wahyu, B. Novala, D. Muhammad, and A. N. Khudori, "Performance Analysis of Sorting Algorithm in Student Data Processing," *JESICA J. Enhanc. Stud. Informatics Comput. Appl.*, vol. 2, no. 2, pp. 52–61, 2025.
- [10] M. Al Nouri, M. R. Albani, M. H. Naibaho, and A. B. Subuh, "Rancang Bangun Sistem Manajemen Nilai Siswa Menggunakan PHP dan MySQL," vol. 2, no. 2, pp. 23–25, 2025.
- [11] N. A. Nasution, N. P. Nainggolan, Z. I. Surbakti, and A. Perdana, "Pengaruh Karakteristik Data Terhadap Performa Algoritma Sorting," *JRIIN J. Ris. Inform. dan Inov. Vol.*, vol. 3, no. 12, pp. 3049–3057, 2026.
- [12] A. Wijaya, N. Aidilia, O. Mevia, R. A. Reswen, A. K. Al-Abror, and F. Ramadhani, "IMPLEMENTASI ALGORITMA INSERTION SORT DALAM," *J. Stud. Multidisipliner*, vol. 9, no. 10, pp. 199–204, 2025.
- [13] H. Y. Tumanggor, R. M. F. Lubis, M. P. Sianturi, and R. G. Purba, "METODE ALGORITMA BUBBLE SORT , ALGORITMA MERGE SORT DAN ALGORITMA QUICK SORT DALAM PENGUJIAN," *JUTISAL (Jurnal Tek. Inform. Komput. Univers.*, vol. 2, no. 2, pp. 47–58, 2022.
- [14] S. Kumari and Aishwarya, "Literature Study on Analyzing and Designing of Algorithms," *Glob. J. Comput. Sci. Technol. C Softw. Data Eng.*, vol. 23, no. 2, pp. 29–35, 2023.
- [15] M. Rizky, S. Barokah, T. Saputra, and T. Sutabri, "Perbandingan Kinerja Algoritma Bubble Sort dan Insertion Sort dalam Pengurutan Data Penjualan UMKM," *MIFORTEKH (Jurnal Manaj. Inform. Teknol.*, vol. 5, no. 1, pp. 184–195, 2025.
- [16] A. Syahputra, "ALGORITMA SELECTION SORT ' ' IMPLEMENTASI ALGORITMA SELECTION SORT UNTUK PENGURUTAN NILAI IPK MAHASISWA UNIVERSITAS POTENSI UTAMA ," *J. Tek. Inform. Kaputama*, vol. 6, no. 2, pp. 390–398, 2022.
- [17] M. F. Abdullah, I. Hafiza, R. Wahyuni, and A. Syahputra, "Penggunaan Algoritma Bubble Sort dalam Pengurutan Nomor Induk Mahasiswa," *J. Ilmu Komput.*, vol. 2, no. 1, pp. 14–19, 2023.